

Oscillatory Neural Network Implementation To Build an Obstacle Avoidance Mobile Robot

Riya Nagare

Electrical Engineering

Indian Institute of Technology Gandhinagar

Gandhinagar, India

riya.nagare@iitgn.ac.in

Mandava Manaswi

Electrical Engineering

Indian Institute of Technology Gandhinagar

Gandhinagar, India

mandava.manaswi@iitgn.ac.in

Abstract—In today's world, the rapidly increasing volume of data generated by edge devices necessitates the development of new algorithms that can reduce both computational time and power consumption. Conventional AI algorithms prove inefficient in addressing these challenges. To overcome these limitations, we employ Oscillatory Neural Networks (ONNs) for obstacle detection by processing sensor data and subsequently generating movement directions for a robotic system. In this approach, two cascaded ONNs are utilized—one dedicated to obstacle detection and the other to computing the appropriate direction. The total latency for generating the final direction is reported to be 108 microseconds.

Index Terms—Edge devices, Oscillatory Neural Networks, Obstacle Avoidance

I. INTRODUCTION

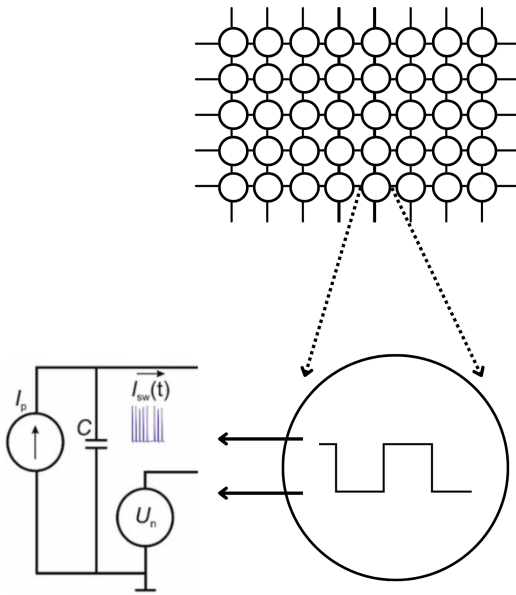


Fig. 1. Oscillatory Neural Network

Edge devices collect large amounts of data from the environment. This demands more time and power due to Von Neumann bottleneck. To address this, we use Oscillatory Neural Networks (ONNs), which are inspired by biological neurons and support parallel processing. ONNs have Non-Von

Neumann architecture (In-memory computing). This makes them faster and power-efficient—ideal for our edge computing project: an obstacle-avoiding mobile robot built using ONNs.

- ONNs mimic biological neurons and support parallel computation, reducing decision latency.
- Due to their non-Von Neumann architecture, ONNs eliminate data transfer overhead between memory and processing, overcoming the Von Neumann bottleneck.
- The robot uses two ONNs, one for obstacle sensing and one for decision-making, to derive direction commands in real time.

II. BACKGROUND AND LITERATURE REVIEW

A. Basics of ONN

ONNs consist of an array of interconnected units (neurons) with weighted synapses. Each neuron operates based on its oscillation phase, and neurons adjust their phases iteratively based on interactions with neighbours. The network evolves toward a minimum energy state, which represents the output pattern.

B. Basic ONN Structure

This figure illustrates the step-by-step working of a digital Oscillatory Neural Network (ONN) for pattern recognition across 8 neurons as an example:

- **Hebbian Learning Rule:** We calculate an 8x8 weight matrix based on the phase states of 8 neurons using the Hebbian learning rule.
- **Output Signal Calculation:** Based on the given phase, the nout signal is generated using a system of flip-flops and multiplexers. The resulting nout signal oscillates between 1 and 0.
- **Input Signal Calculation:** The nin value is computed using the provided formula. Here, W_{ij} represents the weights connected to neurons with $nout = 1$, while W_{ik} represents weights for neurons with $nout = 0$.
- **Phase Alignment:** The phase difference is either added or subtracted depending on which signal edge appears first. If the rising edge of nin appears first, the phase shift is added ($+\Delta\phi$); otherwise, it is subtracted ($-\Delta\phi$).
- **Final Output Declaration:** To determine the final output, the result from the ONN is compared with all trained

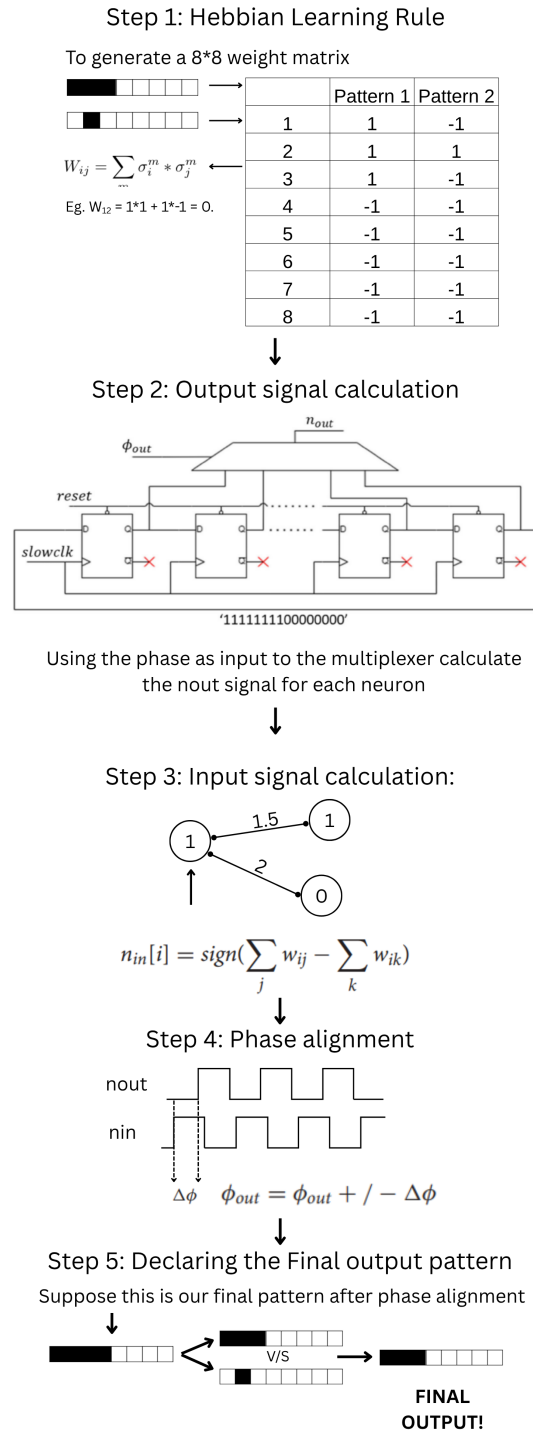


Fig. 2. ONN structure

patterns. The pattern with the smallest phase difference is selected as the output.

III. METHODOLOGY

Overview diagram of the whole project.

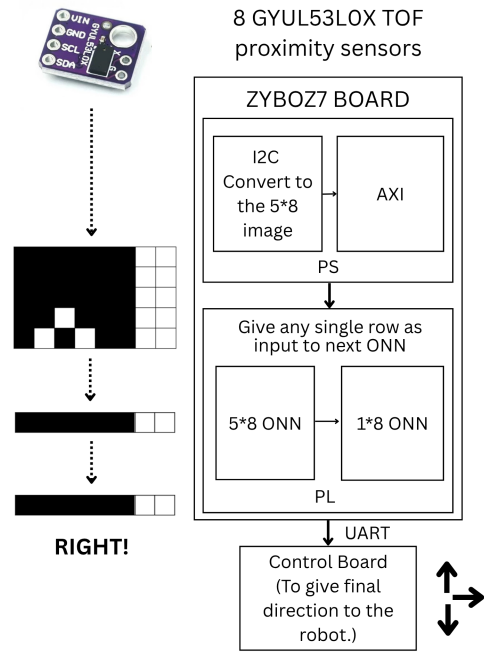


Fig. 3. Overview Diagram

A. PL Part

Trained on 256 exhaustive patterns representing all possible black-and-white column combinations from the 8 TOF proximity sensors. Trained on 16 patterns corresponding to basic navigation commands (e.g., left, forward, right). The output of the 5*8 ONN is a row vector that serves as the input to the 1*8 ONN.

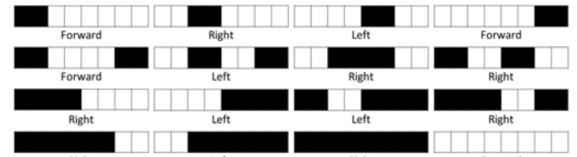


Fig. 4. Overview Diagram

B. PS Part

First Create a Block diagram to integrate the PS and PL through AXI GPIO blocks. Then integrate the sensors through I2C communication protocol and convert their data to 5*8 matrix to give it to the AXI.

C. UART

Finally give the direction output on PMOD and then connect it to the microcontroller Arduino UNO through which motors will get the direction through motor driver.

Name	Slice LUTs (53200)	Slice Registers (109400)	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Bonded IOB (125)	Bonded IOPADs (130)	BUFGCTRL (32)
design_2_wrapper	46718	4616	92	46	12812	46716	2	3	130	5
design_2_j (design_2)	46718	4616	92	46	12812	46716	2	0	0	5

Fig. 5. Utilization Report

IV. RESULTS AND ANALYSIS

A. Vivado Reports

Time required for the cascaded ONNs to reach a direction in simulation is 108 us.

B. ONN Reports

Input to the 5*8 ONN	First row is one of the output row of 5*8 ONN which goes as input to the 1*8 ONN. Second row is the output from 1*8 ONN which corresponds to the direction mentioned below.
	 Forward
	 Forward
	 Right
	 Forward
	 Forward
	 Forward
	 Left
	 Right

Fig. 6. ONN Report

C. Analysis

- The output from the first ONN (5×8) performs as expected, since it has been trained on an exhaustive set of patterns covering all possible input combinations. However, the second ONN, which is trained on only 16 patterns, demonstrates reduced accuracy. As a result, there are certain cases where it produces incorrect or deviated outputs.

- Upon comparing the resource utilization report of our design with that of the referenced research paper, we observe that the number of LUTs used in our implementation is significantly higher. This is primarily because our entire processing—up to the final direction output—has been implemented in hardware using Vivado, whereas the research paper [1] offloads certain computational functions to the Processing System (PS) side.

REFERENCES

- [1] M. Abernot, H. Amara, T. Gil, and A. Todri-Sanial, "Oscillatory Neural Network for Edge Computing: A Mobile Robot Obstacle Avoidance Application," 2022 IEEE International Conference on Metrology for Extended Reality, Artificial Intelligence and Neural Engineering (MetroXRINE), vol. 12, pp. 181–186, Oct. 2022, doi: <https://doi.org/10.1109/metroxraine54828.2022.9967581>.
- [2] A. Velichko, Maksim Belyaev, and Petr Boriskov, "A Model of an Oscillatory Neural Network with Multilevel Neurons for Pattern Recognition and Computing," Electronics (Basel), vol. 8, no. 1, pp. 75–75, Jan. 2019, doi: <https://doi.org/10.3390/electronics8010075>.